

Practical 2 Report, Natural Language Processing 1

Alisia Baielli (15823725)

University of Amsterdam

alisia.baielli@student.uva.nl

John Yao (15667839)

University of Amsterdam

john.yao@student.uva.nl

1 Introduction

1.1 Task

Sentiment classification for documents is a classic task in natural language processing. The task is to predict the sentiment of movie reviews. Every document is assigned a sentiment score from 1 to 5 (1 is a strong negative, 2 is a weak negative, 3 is neutral, 4 is a weak positive, and 5 is a strong positive). All experiments in this work use the Stanford Sentiment Treebank (SST) dataset (Socher et al., 2013), consisting of tree representations of sentences that have a sentiment label at each node.

1.2 Research Questions

The goal of this paper is to analyze several approaches to sentiment classification by answering the following questions: 1) How important is word order? 2) Do tree structures improve model performance? 3) How does sentence length affect performance? 4) When using a tree structure, does including various sub-trees as independent data points improve model performance? For our additional question, 5) When using tree LSTMs, how does the N-ary tree form compare to the Child-Sum tree form? Answering these questions helps us build better representations. If incorporating word order boosts performance (question 1), later models can try to do the same. The same logic holds for exploring if leveraging structure in the form of trees is important (question 2). For question 3, understanding how sentence length affects performance helps build models that are as effective on long passages as short ones. Finally, exploring questions 4 and 5 give us more information about the tree-based approach to language processing.

1.3 Literature

Previous works have not directly answered (1), but various experimental results indicate that including word order in models increases performance. Re-

current neural networks (Elman, 1990) implicitly encode word order when processing language. Subsequent RNN iterations such as Long Short Term Memory (LSTM) networks (Hochreiter, 1997) also account for word order. They represent an improvement in various language processing tasks over models that do not account for word order, such as bag of words (BOW) (Pang, 2002). Question (2) is addressed in the context of Tree LSTMs, independently investigated in three separate works (Tai et al., 2015), (Le and Zuidema, 2015), (Zhu et al., 2015). Results in these papers reveal improved performance over classic LSTMs, indicating a benefit in structured use cases. Question 3) is partially addressed in the literature in the performance improvement of LSTMs over RNNs as LSTMs were designed to model long range dependencies. However, the original LSTM paper does not address its performance between long and short sentences but rather whether information from several sentences in the past is remembered. Furthermore, comparisons between long and short sentences are not done for any other models. Questions 4) and 5) are not addressed directly in the literature.

1.4 Approach

We answer the question of whether word order matters (question 1), by implementing a series of models that do not account for it, and comparing the results to models that do. We start with the BOW approach created by (Pang, 2002). The BOW model does not factor in word order when processing sentences, due to the order-invariance of the summation step. We further implement "continuous bag-of-words" (CBOW), where the dimension size of the embedding space is increased, and "Deep CBOW", where multiple layers are used for embedding. Next, we developed a series of LSTM models to represent word order aware models. For comparison, we used the same GloVe (Pennington et al., 2014) pre-trained word embeddings for both

the LSTM models and the Deep CBOW model. To answer question 2), we implement both the N-ary and Child-Sum Tree LSTMs described in (Tai et al., 2015) and compare the results to classic LSTMs. We address question 3) by separating sentences longer than the median (18 tokens) as long, and shorter than as short. We then compare across all of our models their respective performances. We also examined the results of augmenting the data with all subtrees of the data using the same Tree LSTM models, answering question 4). Finally, we compared the results of the N-ary and Child-sum Tree LSTM experiments to answer question 5).

1.5 Findings

To answer question 1), we found a significant improvement in LSTM models over our best BOW model, Deep CBOW with GloVe embeddings, as our baseline LSTM achieved 46.82% test accuracy while PT-DCBOW achieved 44.25%. For question 2), we note a further 2% improvement of Tree LSTMs over classic LSTMs, at 48.39% test accuracy. In answering question 3), we showed that short sentences were much easier to classify than long ones and averaged 1- 10% improvements in test accuracy. We did not notice a significant change when including subtrees in our Tree LSTM implementations, answering question 4). Finally, we found that the Child-Sum implementation of Tree LSTMs showed a slight decrease in performance compared to the N-ary trees.

2 Background

2.1 Word Embeddings

In order to create a space where words can be numerically compared in meaning and combined, the first step in all experiments is embedding the words in a vector space. These embeddings are optimized during the training process, using backpropagation through gradient descent with cross-entropy loss. In BOW models, the aggregation from word to sentence is a summation of word representations.

2.2 RNN and LSTM

RNNs maintain a hidden representation for each word by including the representation of the previous in the calculation for the current. In this way, word ordering in a sentence is respected. At every word, we apply the same weights to each token to adjust the hidden representation at that time point, and the final representation is obtained

at the end of the sentence. Classic RNNs have the problem of vanishing and exploding gradients due to how the backpropagated gradients are multiplied by the weight matrix at each backwards step and thus can either move towards zero or infinity if not carefully managed. In this work, we use LSTMs which address this issue by including a hidden cell state responsible for retaining and dropping information from early in the sequence through a series of gates. The LSTM formula is listed in the appendix. Specifically, it is important to note that in the final summation 1, the representation is scaled by the input and forget gates. These gates provide a mechanism by which the model can learn to forget inputs that do not affect the overall sentiment and retain important ones.

2.3 Syntax Trees

Language can also be represented as syntax trees, modelling the parts of speech of each sentence. The sentences in SST are syntax tree representations, with sentiment labels at each node and we process these with Tree LSTMs.

2.4 Tree LSTM

We prepare two versions of the Tree LSTM in these experiments, both described in (Tai et al., 2015). The core idea in Tree LSTMs is to create a representation for each token from multiple tokens, rather than just the previous. This creates connections between important parts of a sentence more directly. This process is done recursively until the root of the tree is reached, which is then used as the representation for the sentence.

2.4.1 N-ary Trees

N-ary trees provide each child with separate weights, then summed and used as input for the gates (equations 7). While this limits to a fixed number of children, it has more control over how it processes each child and specific orderings of children are respected. For example, if an input tree is constructed respecting word order (i.e. a left child is always composed of elements to the left of elements of the right child), then the network can learn to assign more weight to left children.

2.4.2 Child Sum Tree

Child Sum Trees sum the child representations before applying a linear layer. This is described in 13. Child Sum Trees are favorable when the number of children is variable at each node, and when it is

desirable to have the same weight for each child of a node.

3 Models

3.1 BOW

All of our BOW models were implemented with a vector lookup for each token, and then passed through linear layers to get to a final embedding where each dimension represents a separate sentiment class. Then each token in a sentence is summed and the highest index in the resulting sequence is the sentiment of the sentence. The process repeats for the document.

3.2 LSTM

Our LSTM models maintain a hidden state h_j and a cellular state c_j for each node in a sentence j . These states are recurrently updated from left to right in a sentence, using the formulas described in the previous section. The final representation for a sentence is computed by mapping the hidden state of the last token to a 5-dimensional embedding, and the largest is chosen as the class. Dropout layers are applied to reduce overfitting. We train the LSTM per sentence, rather than per document.

3.3 Tree LSTM

Our Tree LSTMs train using the same process as regular LSTMs, except that we use the tree structure per sentence in the SST and recurse upwards from the leaf nodes combining left and right children per node.

4 Experiments

4.1 Setup

The SST dataset is split into 8,544 training, 1,101 validation, and 2,210 test datasets. Pre-trained GloVe 840B 300d embeddings are used to initialize word representations. We divide the test set into short and long sentences to assess the model performance with respect to sentence length.

4.2 Hyperparameters and Evaluation

All models are trained using the Adam optimizer and cross-entropy loss. We employ early stopping based on performance on the validation set from previous runs to determine the number of training iterations and other hyperparameters, the details of which are in the appendix. Models are evaluated on the validation set at regular intervals during training, with the evaluation frequency adjusted based

on the total number of iterations. After training, the model achieving the highest validation accuracy is selected. To enhance robustness against random initialization, we repeat this procedure for three different seeds. To evaluate the performance of our sentiment classification models, we use accuracy; it represents the percentage of sentences in the test set whose sentiment was correctly classified. We report the mean test accuracy and its standard deviation across three runs to assess performance and quantify variance.

5 Results and Analysis

Model	Acc.(%)	Std. Dev.
BOW	0.2409	0.0041
CBOW	0.3433	0.0129
DCBOW	0.3688	0.0101
PT-DCBOW	0.4425	0.0004
LSTM	0.4682	0.0121
Batch-LSTM	0.4685	0.0034
Tuned Batch-LSTM	0.4483	0.0090
TLSTM	0.4839	0.0027
CS-TLSTM	0.4691	0.0030

Table 1: Accuracy of different models on our dataset.

5.1 Word Order Importance

Table 1 lists the accuracy of various sentiment classification models. As expected, more complex models achieve higher accuracy as they are more able to capture richer information from the data. BOW models, which do not consider word order, exhibit the lowest accuracy. This is because they do not capture structural dependencies needed to understand sentiment. Deep CBOW, introducing non-linearity through hidden layers, slightly improves accuracy and captures more complex representations of words. With pre-trained word embeddings, PT Deep CBOW yields a significant increase in performance compared to Deep CBOW, which learns its embeddings from scratch on a smaller dataset. GloVe embeddings capture semantic information from a vast corpus of text and provide a richer and more informative initial representation. LSTM models capture long-range dependencies and word order, improving performance by processing words sequentially. They perform better, indicating that word order is important for sentiment classification.

5.2 Including Tree Structure

Tree LSTMs leverage the hierarchical structure of sentences, achieving slightly better performance

than standard LSTM models, with TLSTM obtaining the highest accuracy. This answers question 2). Their architecture captures syntactic relationships between phrases and clauses, providing richer sentence representations.

5.3 Accuracy across Sentence Lengths

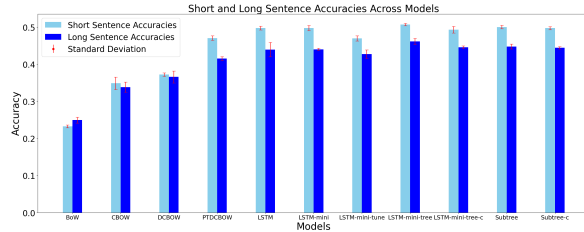


Figure 1: Accuracy across sentence lengths.

We analyze accuracy across sentence lengths (Figure 1) and find that longer sentences are harder to classify accurately. BOW models, which disregard word order, show minimal difference in accuracy between short and long sentences. Pre-trained embeddings (PT-DCBOW) improve overall performance as expected but show a larger accuracy gap between short and long sentences. This is possibly because pre-trained embeddings on large, general-domain datasets are good at recognizing sharp sentiment signals common in short sentences. While accuracy still improves for longer sentences, the improvement is smaller due to their noisier sentiment signals. Further experimentation is needed. This phenomenon does not occur with BOW models because their initial embeddings do not learn properly for both short and long sentences. LSTM models perform worse on longer sentences due to sequential processing, which can cause earlier information to lose importance in the sentence. Tree LSTMs reduce this accuracy gap by leveraging their hierarchical structure, enabling them to capture relationships between parts of a sentence more effectively. By breaking down long sentences into smaller units, they mitigate the forgetting effect and reliance on sequential dependencies.

Model	Acc.(%)	Std. Dev.
TLSTM	0.4738	0.0048
CS-TLSTM	0.4704	0.0009

Table 2: Tree LSTM models with subtrees.

5.4 Supervising Sentiment at Each Node

Table 2 shows the accuracy of TLSTM and CS-TLSTM trained on the extended dataset with node-

level supervision. Surprisingly, their performance remains similar to models trained on the standard dataset. We expected that supervising sentiment at each node would provide finer-grained training signals, enabling better representations of sentiment at different levels of the sentence. We posit that this form of augmentation provides little new information, akin to repeating data.

5.5 N-ary LSTM vs ChildSum

The binary Tree LSTM is built for trees with a fixed number of children, assigning different parameters to each child. This captures sentence structure, such as the order of words and their relationships. The Child-Sum Tree LSTM combines all children with equal weight, giving it more flexibility for trees with varying numbers of children but less accurate for tasks that depend on structure. From our results, we see that the N-ary version is superior on our dataset. This is expected, as all sentences in our data are fixed binary trees. A model with the capacity to distinguish left and right children would unlikely perform worse than one that did not.

6 Conclusions

From our experiments, we answered our research questions and we deduce that word order and sentence structure are important in sentiment analysis. Models that leverage these aspects, such as LSTM and Tree LSTM, achieve higher performance compared to simpler models like BOW. Pre-trained word embeddings provide rich semantic representations, leading to improved accuracy. However, they exhibit limitations with longer sentences. The reason for this is still not clear, and further investigation is needed. LSTMs perform worse as sentence length increases, possibly due to the fact that earlier information is gradually lost in sequential processing. Tree LSTMs mitigate this by leveraging hierarchical sentence structures and achieve the highest accuracy overall. These results support findings from previous works, confirming that models incorporating order, context and sequential information perform better. Future work could explore transformer-based models (Prottasha et al., 2022) (Abdaoui and Dutta, 2023), good at capturing long-range dependencies and context, or investigate further fine-tuning of the embeddings (Perikos and Diamantopoulos, 2024) to address challenges with longer sentences.

References

- Amine Abdaoui and Sourav Dutta. 2023. Attention over pre-trained sentence embeddings for long document classification. *arXiv preprint arXiv:2307.09084*.
- Jeffrey L Elman. 1990. Finding structure in time. *Cognitive science*, 14(2):179–211.
- S Hochreiter. 1997. Long short-term memory. *Neural Computation MIT-Press*.
- Phong Le and Willem Zuidema. 2015. Compositional distributional semantics with long short term memory. *arXiv preprint arXiv:1503.02510*.
- B Pang. 2002. Thumbs up? sentiment classification using machine learning techniques. In *Proceedings of the ACL-02 conference on Empirical methods in natural language processing/10*.
- Jeffrey Pennington, Richard Socher, and Christopher D Manning. 2014. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543.
- Isidoros Perikos and Athanasios Diamantopoulos. 2024. Explainable aspect-based sentiment analysis using transformer models. *Big Data and Cognitive Computing*, 8(11):141.
- Nusrat Jahan Prottasha, Abdullah As Sami, Md Kowsher, Saydul Akbar Murad, Anupam Kumar Bairagi, Mehedi Masud, and Mohammed Baz. 2022. Transfer learning for sentiment analysis using bert based supervised fine-tuning. *Sensors*, 22(11):4157.
- Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1631–1642.
- Kai Sheng Tai, Richard Socher, and Christopher D Manning. 2015. Improved semantic representations from tree-structured long short-term memory networks. *arXiv preprint arXiv:1503.00075*.
- Xiaodan Zhu, Parinaz Sobihani, and Hongyu Guo. 2015. Long short-term memory over recursive structures. In *International conference on machine learning*, pages 1604–1612. PMLR.

A Appendix

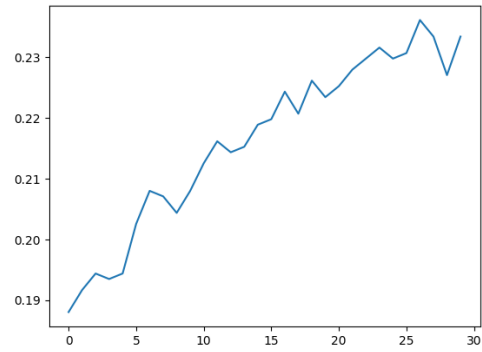


Figure 2: BOW Accuracy.

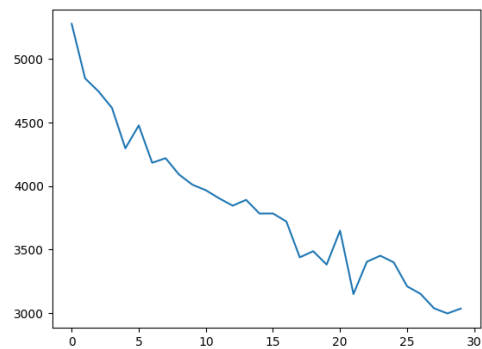


Figure 3: BOW Loss.

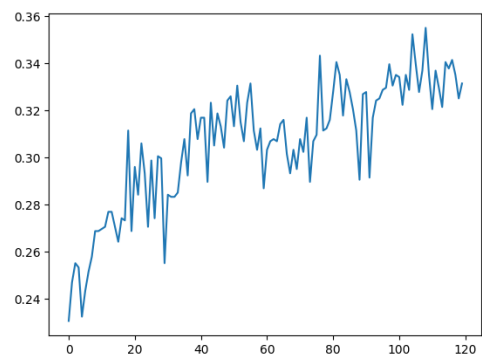


Figure 4: CBOW Accuracy.

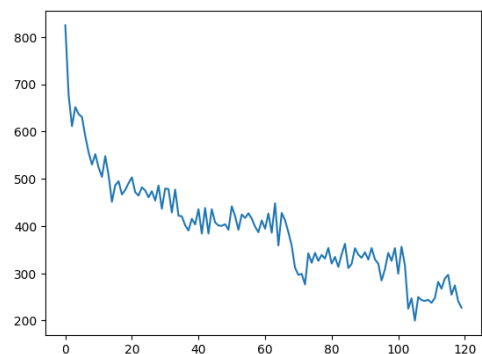


Figure 5: CBOW Loss.

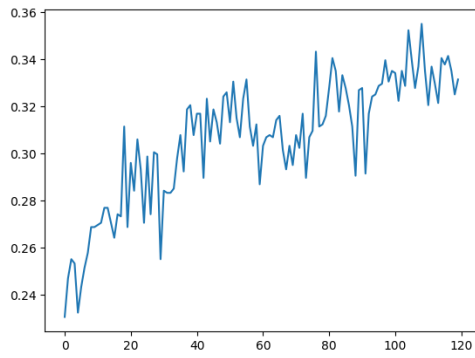


Figure 6: DCBOW Accuracy.

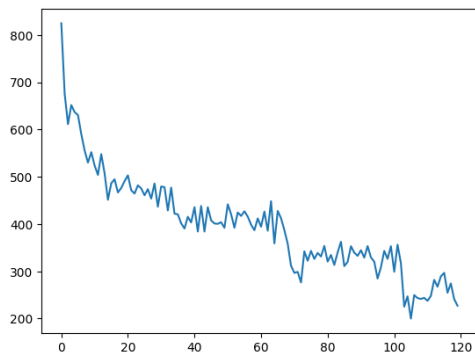


Figure 7: DCBOW Loss.

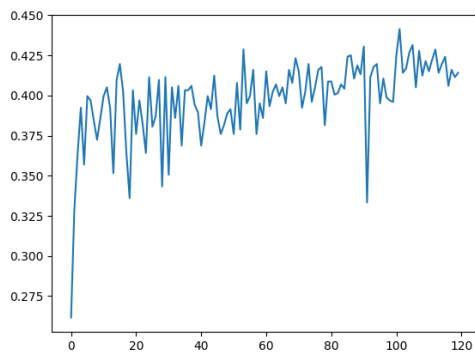


Figure 8: PT-DCBOW Accuracy.

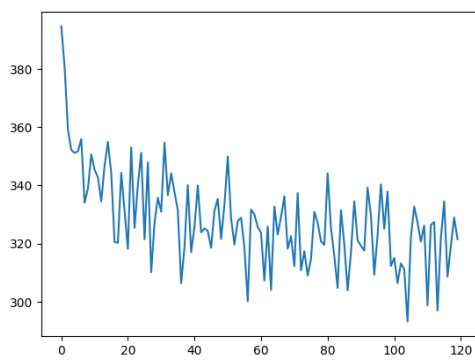


Figure 9: PT-DCBOW Loss.

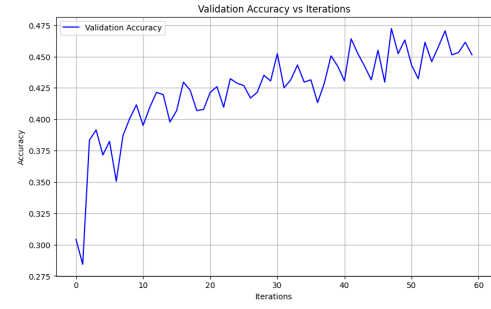


Figure 10: LSTM Accuracy.

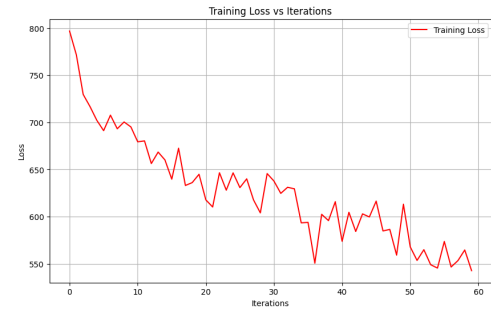


Figure 11: LSTM Loss.

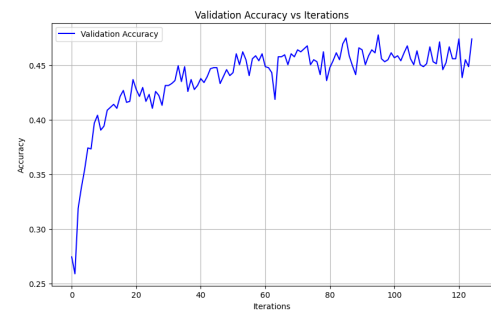


Figure 12: LSTM MINI Accuracy.

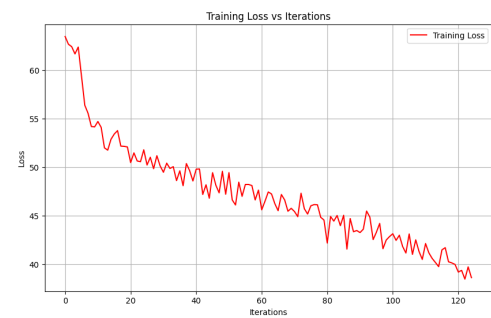


Figure 13: LSTM MINI Loss.

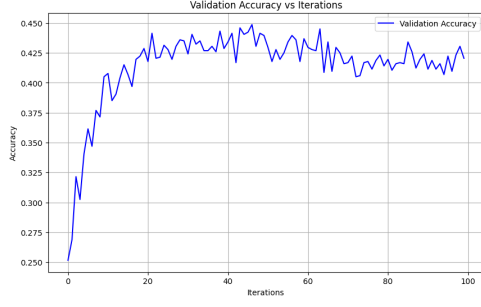


Figure 14: LSTM TUNE Accuracy.

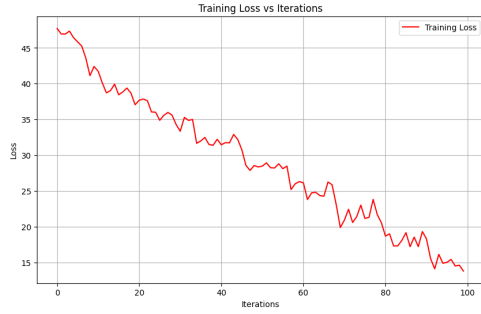


Figure 15: LSTM TUNE Loss.

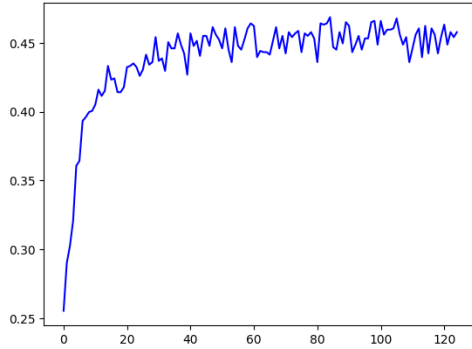


Figure 16: TREE-LSTM Accuracy.

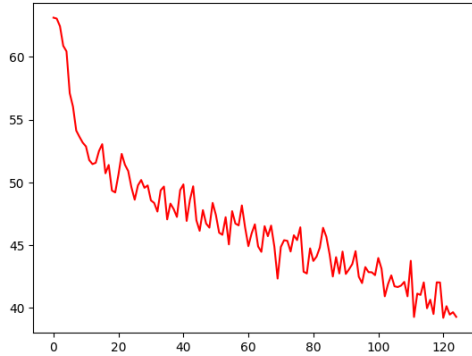


Figure 17: TREE-LSTM Loss.

LSTM formula:

$$i = \sigma(W_{ii}x + b_{ii} + W_{hi}h + b_{hi}) \quad (1)$$

$$f = \sigma(W_{if}x + b_{if} + W_{hf}h + b_{hf}) \quad (2)$$

$$g = \tanh(W_{ig}x + b_{ig} + W_{hg}h + b_{hg}) \quad (3)$$

$$o = \sigma(W_{io}x + b_{io} + W_{ho}h + b_{ho}) \quad (4)$$

$$c' = f * c + i * g \quad (5)$$

$$h' = o \tanh(c') \quad (6)$$

N-ary Tree LSTM formula:

$$i = \sigma(W_{ii}x + b_{ii} + \sum_{l=1}^N W_{hkil}h_l + b_{hi}) \quad (7)$$

$$f_k = \sigma(W_{if}x + b_{if} + \sum_{l=1}^N W_{hkfl}h_l + b_{hf}) \quad (8)$$

$$g = \tanh(W_{ig}x + b_{ig} + \sum_{l=1}^N W_{hkgil}h_l + b_{hg}) \quad (9)$$

$$o = \sigma(W_{io}x + b_{io} + \sum_{l=1}^N W_{hkol}h_l + b_{ho}) \quad (10)$$

$$c' = \sum_{l=1}^N f_l * c_l + i * g \quad (11)$$

$$h' = o \tanh(c') \quad (12)$$

Child Sum Tree LSTM formula:

$$\tilde{h} = \sum_{k \in C} h_k \quad (13)$$

$$i = \sigma(W_{ii}x + b_{ii} + W_{hi}\tilde{h} + b_{hi}) \quad (14)$$

$$f_k = \sigma(W_{if}x + b_{if} + W_{hf}h_k + b_{hf}) \quad (15)$$

$$g = \tanh(W_{ig}x + b_{ig} + W_{hg}\tilde{h} + b_{hg}) \quad (16)$$

$$o = \sigma(W_{io}x + b_{io} + W_{ho}\tilde{h} + b_{ho}) \quad (17)$$

$$c' = \sum_{k \in C} f_k * c_k + i * g \quad (18)$$

$$h' = o \tanh(c') \quad (19)$$

Hyperparameters

The batch size is 1 for the BOW model and 25 for all other models. BOW models use a learning rate of 0.0005, while LSTM models use 2e-4. For word embeddings, BOW use a size equal to the number

of sentiment classes (5), while other models use 300-dimensional embeddings. PT Deep CBOW and LSTM models initialize with pre-trained 300-dimensional GloVe vectors. Deep CBOW use a hidden layer with size 100, LSTM with size 168, and Tree-LSTM with size 150.